

Credit Scoring Framework

Line-by-Line Codebase Walkthrough | Generated 2026-03-19

Contents

1. scorecard_builder.py — Core scorecard logic
2. monitoring.py — Performance metrics
3. reject_inference.py — Reject inference engine
4. recalibration.py — Model recalibration
5. main.py — Entry point & orchestration
6. Architecture summary

1. scorecard_builder.py

Lines 1–3 — Module header & debug print

```
# Scorecard Builder
# Includes WOE, binning, logistic regression, score scaling
print("Scorecard Builder Module")
```

What: A comment describing the module, followed by a `print` statement.

Why: The print was added during early development to confirm the module was being imported correctly.

How: When Python imports this file, all top-level code runs immediately — so this print fires the moment you do `from scorecard_builder import ScorecardBuilder`.

Issue: This is debug noise. In production it should be removed or replaced with `logging.getLogger(__name__)`.

Lines 5–8 — Imports

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from optbinning import OptimalBinning
```

What: Brings in four external libraries.

Import	What it provides	Why it's needed here
<code>pandas</code>	DataFrames, tabular data manipulation	Holding and slicing the input dataset
<code>numpy</code>	Fast array math	Log/dot product math in the scoring formula
<code>LogisticRegression</code>	Linear classification model	The actual credit risk model
<code>OptimalBinning</code>	Supervised binning + WOE/IV calculation	Industry-standard binning for scorecards

How: Python's import system loads these from installed packages in the `.venv` environment.

Lines 10–15 — Class definition & constructor

```
class ScorecardBuilder:

    def __init__(self, target):
        self.target = target
        self.model = None
        self.binning_models = {}
```

What: Defines the `ScorecardBuilder` class with three instance variables.

Variable	Initial value	Purpose
<code>self.target</code>	string (e.g. <code>"default_flag"</code>)	Name of the column holding 0/1 default labels
<code>self.model</code>	<code>None</code>	Will hold the fitted <code>LogisticRegression</code> after <code>train()</code>
<code>self.binning_models</code>	<code>{}</code> (empty dict)	Will hold one <code>OptimalBinning</code> object per feature after <code>transform_woe()</code>

Why: Using a class lets the scorecard carry its state between steps. You train once, then call `score()` repeatedly on new data — `self.model` and `self.binning_models` persist across those calls.

How: `__init__` is Python's constructor. It runs automatically when you write `builder = ScorecardBuilder(target="default_flag")`.

Lines 17–20 — `calculate_iv()`

```
def calculate_iv(self, df, feature):
    optb = OptimalBinning(name=feature, dtype="numerical")
    optb.fit(df[feature], df[self.target])
    return optb.binning_table.build()["IV"].sum()
```

What: Computes the **Information Value (IV)** of a single feature.

Why: IV measures how well a variable separates good borrowers from bad ones. It is the standard feature-selection metric in credit scoring.

IV range	Predictive power
< 0.02	Useless
0.02 – 0.1	Weak
0.1 – 0.3	Medium
> 0.3	Strong

How:

- `OptimalBinning(name=feature, dtype="numerical")` — Creates a binning object. Applies supervised monotone optimal binning to find best cut points.
- `optb.fit(df[feature], df[self.target])` — Feeds feature values and 0/1 labels to find optimal bin boundaries.
- `optb.binning_table.build()` — Produces a per-bin summary with WoE, IV columns.
- `["IV"].sum()` — Sums per-bin IV contributions to get total feature IV.

IV_bin = (% Goods in bin - % Bads in bin) × ln(% Goods / % Bads)
 Total IV = Σ IV_bin across all bins

Lines 22–33 — `select_features()`

```
def select_features(self, df, threshold=0.02):
    iv_dict = {}
    for col in df.columns:
        if col == self.target:
            continue
        try:
            iv_dict[col] = self.calculate_iv(df, col)
        except:
            continue
```

```
iv_df = pd.DataFrame.from_dict(iv_dict, orient="index", columns=["IV"])
return iv_df[iv_df["IV"] > threshold].index.tolist()
```

What: Loops over every column, calculates IV, and returns only those with IV > `threshold` (default 0.02).

How:

- `iv_dict = {}` — Accumulator: `{feature_name: iv_value}`.
- `if col == self.target: continue` — Skips the target column (can't use outcome as predictor).
- `try / except: continue` — Silently skips features that cause errors (constant column, all-null, etc.).
- `pd.DataFrame.from_dict(iv_dict, orient="index")` — Converts dict to DataFrame with feature names as index.
- `iv_df[iv_df["IV"] > threshold].index.tolist()` — Returns names of features that pass the IV threshold.

Issue: Bare `except:` catches everything including `KeyboardInterrupt`. Should be `except Exception:` at minimum, and should log the skipped feature name for debugging.

Lines 35–45 — `transform_woe()`

```
def transform_woe(self, df, features):
    woe_df = pd.DataFrame()

    for col in features:
        optb = OptimalBinning(name=col, dtype="numerical")
        optb.fit(df[col], df[self.target])
        self.binning_models[col] = optb
        woe_df[col] = optb.transform(df[col], metric="woe")

    woe_df[self.target] = df[self.target]
    return woe_df
```

What: Replaces raw feature values with their **Weight of Evidence (WOE)** equivalents, and stores the fitted binning objects.

Why WOE?

- Handles non-linearity — each bin maps to a single WOE value
- Handles outliers — extreme values fall into edge bins
- Makes logistic regression coefficients more interpretable
- Encodes the relationship between the feature and target directly

$WOE = \ln(\text{Distribution of Goods} / \text{Distribution of Bads})$
 $= \ln(\% \text{ of total Goods in bin} / \% \text{ of total Bads in bin})$

Positive WOE bin → more goods than bads → lower credit risk

Negative WOE bin → more bads than goods → higher credit risk

How:

- `self.binning_models[col] = optb` — **Critical:** stores the fitted binner so it can be applied to new data at scoring time.
- `optb.transform(df[col], metric="woe")` — Replaces raw values with each value's bin WOE score.
- `woe_df[self.target] = df[self.target]` — Carries the target column through for immediate model training.

Issue: Re-fits OptimalBinning on the same data used for training — mild data leakage. Binning should be fit on a training set and only applied (`.transform()`) to validation/test sets.

Lines 47–57 — `train()`

```
def train(self, df):
    features = self.select_features(df)
    woe_df = self.transform_woe(df, features)

    X = woe_df[features]
    y = woe_df[self.target]

    self.model = LogisticRegression(max_iter=1000)
    self.model.fit(X, y)

    return self.model, features
```

What: Orchestrates the full training pipeline: feature selection → WOE transformation → logistic regression fitting.

How:

- `select_features(df)` → list of features with IV > 0.02
- `transform_woe(df, features)` → WOE DataFrame + saves binning models
- `X = woe_df[features]` — feature matrix (WOE-transformed)
- `y = woe_df[self.target]` — target vector (0 = good, 1 = bad)
- `LogisticRegression(max_iter=1000)` — max_iter increased from default 100 because WOE data sometimes takes more iterations to converge
- `return self.model, features` — caller needs both to know which features the model expects

Issue: No train/test split. Model is evaluated on training data, which overstates performance.

Lines 59–65 — `score()`

```
def score(self, X, pdo=50, base_score=600, base_odds=50):
    factor = pdo / np.log(2)
    offset = base_score - factor * np.log(base_odds)

    log_odds = self.model.intercept_[0] + np.dot(X, self.model.coef_[0])
    score = offset + factor * log_odds

    return score
```

What: Converts the logistic regression's log-odds output into a credit score on a human-readable scale (e.g. 300–850) using the industry-standard **PDO (Points to Double the Odds)** convention.

Why:

- At `base_score` (600), the odds of being good are `base_odds` (50:1)
- Every `pdo` (50) additional points doubles the odds of being good

```
factor = PDO / ln(2)
offset = base_score - factor * ln(base_odds)
score = offset + factor * log_odds
```

An increase in `log_odds` of $\ln(2)$ (doubling the odds) adds exactly PDO points to the score.

How:

- `factor = pdo / np.log(2)` — Scaling multiplier. With PDO=50, $\text{factor} \approx 72.1$
- `offset = base_score - factor * np.log(base_odds)` — Pins the scale: at $\text{log-odds} = \ln(50)$, `score = 600`
- `self.model.intercept_[0] + np.dot(X, self.model.coef_[0])` — Manually computes log-odds: bias term + $\sum(\text{weight} \times \text{WOE})$
- `score = offset + factor * log_odds` — Applies PDO scaling to produce the final credit score

2. monitoring.py

Lines 1–6 — Header & imports

```
# Monitoring Engine
# KS, PSI, lender metrics
print("Monitoring Module")

import numpy as np
import pandas as pd
```

What: Same debug-print pattern as `scorecard_builder.py`. Only `numpy` and `pandas` needed — no ML models, just statistics.

Lines 8–15 — `calculate_ks()`

```
def calculate_ks(df, score_col, target_col):
    df = df.sort_values(score_col)

    df["cum_bad"] = df[target_col].cumsum() / df[target_col].sum()
    df["cum_good"] = ((1 - df[target_col]).cumsum()) / (1 - df[target_col].sum())

    ks = np.max(np.abs(df["cum_bad"] - df["cum_good"]))
    return ks
```

What: Calculates the **Kolmogorov-Smirnov (KS) statistic** — the primary performance metric for credit scorecards, measuring how well the score separates defaulters from non-defaulters.

KS	Interpretation
< 0.20	Poor
0.20 – 0.40	Acceptable
0.40 – 0.60	Good
> 0.60	Excellent (check for leakage)

How:

- `df.sort_values(score_col)` — Sorts all applicants lowest to highest score. Essential for cumulative distribution calculation.

- `df[target_col].cumsum() / df[target_col].sum()` — For each score threshold, what fraction of all bads have been captured? Normalised to [0, 1].
- `(1 - df[target_col]).cumsum() / (1 - df[target_col]).sum()` — Same idea for goods: flips bads→0 and goods→1, then cumulative fraction captured.
- `np.max(np.abs(df["cum_bad"] - df["cum_good"]))` — Maximum gap between cumulative bad rate and good rate across all score thresholds. This peak separation is the KS statistic.

Lines 18–27 — `calculate_psi()`

```
def calculate_psi(expected, actual, bins=10):
    breakpoints = np.linspace(0, 1, bins + 1)

    expected_percent = np.histogram(expected, bins=breakpoints)[0] / len(expected)
    actual_percent = np.histogram(actual, bins=breakpoints)[0] / len(actual)

    psi = np.sum((actual_percent - expected_percent) *
                 np.log((actual_percent + 1e-6) / (expected_percent + 1e-6)))

    return psi
```

What: Calculates the **Population Stability Index (PSI)** — measures how much the score distribution has shifted between two periods (e.g. development vs. current production).

PSI	Interpretation
< 0.10	Stable — no action needed
0.10 – 0.25	Minor shift — monitor
> 0.25	Major shift — investigate, consider retraining

How:

- `np.linspace(0, 1, bins + 1)` — Creates 11 evenly spaced points → 10 bin edges. Assumes scores/probabilities are normalised to [0, 1].
- `np.histogram(...)[0] / len(expected)` — Counts per bin, divided by total → proportions summing to 1.
- PSI formula: $\sum (\text{Actual\%} - \text{Expected\%}) \times \ln(\text{Actual\%} / \text{Expected\%})$ — a symmetric divergence measure based on KL divergence.
- `+ 1e-6` — Tiny constant prevents `log(0) = -∞` when any bin is empty.

Issue: Breakpoints are fixed to [0, 1], which only works if inputs are probabilities. Raw credit scores (e.g. 400–750) would need different breakpoints.

Lines 30–46 — `lender_metrics()`

```
def lender_metrics(df):
    results = []

    for lender in df["lender_id"].unique():
        temp = df[df["lender_id"] == lender]

        ks = calculate_ks(temp, "score", "target")
        bad_rate = temp["target"].mean()

        results.append({
            "lender_id": lender,
            "ks": ks,
            "bad_rate": bad_rate,
            "volume": len(temp)
        })

    return pd.DataFrame(results)
```

What: Breaks down model performance by lender — useful in multi-lender or marketplace lending platforms.

Why: A model may perform well overall but poorly for a specific lender whose population is different (different geographies, products, or risk appetite).

How: Iterates over each distinct lender, filters to their sub-population, computes KS, bad rate, and volume, then returns a tidy summary DataFrame. Bad rate = `target.mean()` = proportion of defaults.

3. reject_inference.py

Lines 1–7 — Header & imports

```
# Reject Inference Engine
# Parceling, fuzzy, weighting
print("Reject Inference Module")

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
```

The comment mentions three methods — *Parceling*, *fuzzy*, *weighting* — but only parceling is implemented. The other two are planned future additions.

Lines 9–12 — Class & constructor

```
class RejectInference:

    def __init__(self, features):
        self.features = features
```

What: Stores the list of feature column names.

Why: The `parceling()` method needs to know which columns to use as predictors when training an internal proxy model on approved customers.

Lines 14–32 — `parceling()`

```
def parceling(self, df):
    approved = df[df["approved"] == 1].copy()
    rejected = df[df["approved"] == 0].copy()

    model = LogisticRegression(max_iter=1000)
    model.fit(approved[self.features], approved["target"])

    rejected["pd"] = model.predict_proba(rejected[self.features])[:, 1]

    approved["band"] = pd.qcut(
        model.predict_proba(approved[self.features])[:, 1], 10, labels=False, duplicates="drop"
    )
    rejected["band"] = pd.qcut(rejected["pd"], 10, labels=False, duplicates="drop")

    band_bad_rate = approved.groupby("band")["target"].mean()
```

```

rejected["target"] = rejected["band"].map(band_bad_rate)
rejected["target"] = (np.random.rand(len(rejected)) <
rejected["target"].fillna(approved["target"].mean())).astype(int)

return pd.concat([approved, rejected])

```

What: Infers likely outcomes (good/bad) for rejected applicants using the **parceling (banding) method**.

Why reject inference? A scorecard trained only on approved applicants suffers from **sample selection bias** — rejects were excluded precisely because they looked riskier. Ignoring them means the model never learned from that portion of the population, making it less accurate near the decision boundary.

How — step by step:

1. `approved = df[df["approved"] == 1].copy()` / `rejected = df[df["approved"] == 0].copy()` — Splits data. `.copy()` prevents pandas SettingWithCopyWarning when adding columns.
2. `model.fit(approved[self.features], approved["target"])` — Trains a proxy logistic regression *only on approved customers* (who have known outcomes). Used purely for the banding step.
3. `rejected["pd"] = model.predict_proba(rejected[self.features])[:, 1]` — Scores rejected applicants using the approved-trained model. `[:, 1]` takes the probability of default column.
4. `pd.qcut(..., 10, ...)` — Divides applicants into 10 equal-frequency bands (deciles) by predicted default probability. `duplicates="drop"` handles ties. Applied to both approved and rejected.
5. `band_bad_rate = approved.groupby("band")["target"].mean()` — For each band, computes the observed default rate among approved customers. This is the key lookup table.
6. `rejected["target"] = rejected["band"].map(band_bad_rate)` — Maps each reject's band to the observed bad rate from approved customers in the same band. Gives each reject a *probability* of default.
7. `(np.random.rand(len(rejected)) < rejected["target"].fillna(...)).astype(int)` — Converts the probability to a concrete 0/1 label via a **Bernoulli draw**. If `uniform_random < probability_of_default`, assign bad (1), else good (0). This is the "parceling" — probabilistically parceling rejects into good and bad categories. `.fillna(approved["target"].mean())` handles unmapped bands using the overall bad rate as fallback.
8. `pd.concat([approved, rejected])` — Combines real outcomes (approved) and inferred outcomes (rejected) into one dataset for model retraining.

4. recalibration.py

Lines 1–6 — Header & imports

```
# Auto Recalibration Engine
# Intercept shift, Platt scaling, isotonic regression
print("Recalibration Module")

import numpy as np
```

Only `numpy` needed — pure math, no DataFrames. The comment mentions three methods; only `intercept_shift` is implemented.

Lines 7–17 — `intercept_shift()`

```
def intercept_shift(df):
    expected_pd = df["pd"].mean()
    actual_pd = df["target"].mean()

    adjustment = np.log(actual_pd / (1 - actual_pd)) - np.log(expected_pd / (1 - expected_pd))

    df["log_odds"] = np.log(df["pd"] / (1 - df["pd"]))
    df["log_odds_adj"] = df["log_odds"] + adjustment
    df["pd_calibrated"] = 1 / (1 + np.exp(-df["log_odds_adj"]))

    return df
```

What: Adjusts the model's predicted probability of default (PD) to match observed reality.

Why recalibration? Over time, the real-world default rate drifts from what the model predicts. A model built 2 years ago might predict a 5% bad rate, but today's actual rate is 8%. Intercept shift corrects this without full retraining — it slides all predictions up or down by a constant in log-odds space.

How:

- `expected_pd = df["pd"].mean()` — Model's average predicted default probability.
- `actual_pd = df["target"].mean()` — Actual observed default rate in the same population.
- `adjustment = log(actual_pd / (1 - actual_pd)) - log(expected_pd / (1 - expected_pd))` — Converts both rates to **log-odds**, then takes the difference. This is the exact constant shift needed in log-odds space to align mean prediction with observed rate.
- `df["log_odds"] = np.log(df["pd"] / (1 - df["pd"]))` — Converts each individual predicted PD back to log-odds.

- `df["log_odds_adj"] = df["log_odds"] + adjustment` — Adds the constant shift to every applicant's log-odds.
- `df["pd_calibrated"] = 1 / (1 + np.exp(-df["log_odds_adj"]))` — The **sigmoid function**: converts adjusted log-odds back to a probability in [0, 1]. It is the mathematical inverse of the log-odds function.

$$\sigma(x) = 1 / (1 + e^{-x})$$

5. main.py

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent))

import pandas as pd
from scorecard_builder import ScorecardBuilder
from monitoring import calculate_ks

df = pd.read_csv("data/sample_data.csv")

builder = ScorecardBuilder(target="default_flag")

model, features = builder.train(df)

woe_df = builder.transform_woe(df, features)

df["score"] = builder.score(woe_df[features])

ks = calculate_ks(df, "score", "default_flag")

print("KS:", ks)
```

Lines 1–5 — Path setup

What: Ensures Python can find the modules in the same folder as `main.py`.

Why: When running from a different working directory, Python's module search path may not include the project folder. This line prepends it at position 0 (highest priority).

How: `Path(__file__).parent` resolves to the directory containing `main.py`. `sys.path.insert(0, ...)` puts it first in the search path.

Lines 7–9 — Imports

What: Imports pandas and the two things actually used in this script. `reject_inference` and `recalibration` are not imported — `main.py` demonstrates only the core happy-path pipeline.

Line 11 — Load data

What: Reads the dataset from a hard-coded relative path.

Issue: If run from a different directory, this fails. Better: use `Path(__file__).parent / "data" / "sample_data.csv"`.

Line 13 — Initialise builder

What: Creates a `ScorecardBuilder` instance, telling it the target column is `"default_flag"`.

Line 15 — Train

What: Runs the full pipeline: IV selection → WOE binning → logistic regression. Returns the fitted model and selected feature names.

Line 17 — Re-apply WOE

What: Re-transforms the data to WOE to get values for scoring.

Issue: `train()` internally calls `transform_woe()` but doesn't return the WOE DataFrame. This line re-applies the transformation — calling `OptimalBinning.fit()` a second time on the same data. Wasted computation. A cleaner design would have `train()` return or cache the WOE DataFrame.

Lines 19–23 — Score & evaluate

What: Generates credit scores via PDO scaling and computes KS to measure model performance.

Issue: In-sample KS (training data = evaluation data), so the number will be optimistic. A proper evaluation would use a held-out test set.

6. Architecture Summary

```
main.py
├── ScorecardBuilder.train(df)
│   ├── select_features() → IV via OptimalBinning → keep IV > 0.02
│   └── transform_woe() → WOE via OptimalBinning → fit LogisticRegression
├── ScorecardBuilder.score() → PDO scaling → credit scores
└── calculate_ks() → sweep score thresholds → max cumulative separation

reject_inference.py (standalone)
└── RejectInference.parceling() → proxy model → band mapping → Bernoulli draw

recalibration.py (standalone)
```

```
└─ intercept_shift() → log-odds gap → shift all predictions → sigmoid back to PD
```

```
monitoring.py (standalone / ongoing)
```

```
└─ calculate_ks() → model discrimination
```

```
└─ calculate_psi() → population drift
```

```
└─ lender_metrics() → segment-level performance
```